

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЁТ

по практической работе № 3

«Объектно-ориентированное программирование в Python. Классы, объекты и взаимодействие объектов»

Дисциплина: Технологии разработки приложений на базе фреймворков

Семестр: 5 семестр, 2026–2027 учебный год

Студент: Думлер Виктория

Группа: _____

Преподаватель: Яцун Татьяна Викторовна

Проект: Система управления пользовательскими ролями

Год выполнения: 2026

1. Выполненные материалы Яндекс Практикума

Материалы раздела 03 курса «Бэкенд-разработка» по ООП и декораторам выполнены в объёме методички ПР3. Материалы ПР1 и ПР2 повторно не проходились, но используются в проекте.

Раздел	Выполненные темы
Раздел 03. Объекты и классы; знакомство с ООП	Введение в ООП; классы и объекты; атрибуты и методы; <code>__str__</code> ; принципы ООП; наследование; полиморфизм; инкапсуляция; практики по темам
Раздел 03. Расширенные возможности Python	Декораторы; декораторы для методов класса; практика по теме

2. Доработка индивидуального проекта

Название: Система управления пользовательскими ролями.

Проект ПР2 не переписывался с нуля: словари заменены объектами, меню и сценарии сохранены. Классы вынесены в пакет `models/`.

Основные классы ПР3:

- `Entity` — базовый класс (наследование);
- `User` — пользователь, ссылки на `Role` и `Group`;
- `Role` — роль и список `Permission`, правило доступа ПР1;
- `Permission` — разрешение на действие;
- `Group` — группа (отдел).

Репозиторий: <https://github.com/VikaD2345/-User-role-management-system>

3. Результат работы

Сущности представлены классами с `__init__` и `__str__`. Коллекции: `list[User]`, `list[Role]`, `list[Group]`, `list[Permission]`. JSON загружается в объекты и сохраняет `id` связей. `pytest`: 22 passed; `flake8` без замечаний. README обновлён.

4. Вывод

ПР3 перевела функциональную модель ПР2 в объектную: данные и поведение объединены в объектах, связи заданы композицией, принципы ООП показаны наследованием `Entity`, полиморфизмом `__str__` и инкапсуляцией методов роли. Приложение готово к следующему этапу — Django.

КОНТРОЛЬНЫЕ ВОПРОСЫ

Практическая работа № 3

Думлер Виктория · система управления пользовательскими ролями

Ответы привязаны к коду проекта, чтобы отвечать на защите с демонстрацией файла.

По классам и объектам

1. Что такое класс?

Ответ. Описание типа объектов: какие у них данные и поведение. В проекте классы User, Role, Permission, Group и базовый Entity.

2. Что такое объект?

Ответ. Конкретный экземпляр класса. Иванова А.С. — объект User, её роль «менеджер» — объект Role.

3. Чем класс отличается от объекта?

Ответ. Класс — чертёж (User), объект — экземпляр (user = User(...)). Класс один, объектов может быть много.

4. Что такое экземпляр класса?

Ответ. То же, что объект. users[0] и users[5] — разные экземпляры User с разными атрибутами.

5. Для чего используется __init__?

Ответ. Конструктор: заполняет атрибуты при создании. User.__init__ сохраняет id, имя, возраст, роль и группу.

6. Что такое self?

Ответ. Ссылка на текущий объект внутри метода. self.role — роль именно этого пользователя.

7. Чем атрибут класса отличается от атрибута объекта?

Ответ. Атрибут объекта принадлежит экземпляру: user.age. Атрибут класса общий для всех. У нас данные — атрибуты объекта.

8. Что такое метод объекта?

Ответ. Функция внутри класса, работает с self. user.has_permission(...) проверяет права этого пользователя.

9. Чем метод отличается от обычной функции?

Ответ. Метод связан с объектом и видит его атрибуты. Функция find_users(users, query) работает с коллекцией, поэтому осталась обычной.

10. В каких случаях функцию следует сделать методом класса?

Ответ. Если операция относится к одному объекту и его состоянию. Назначить роль — user.assign_role(). Поиск по списку — функция.

11. Для чего используется __str__?

Ответ. Чтобы `print(user)` печатал понятную карточку, а не адрес в памяти. Каждый класс переопределяет `__str__`.

12. Что происходит при выполнении `print(object)`?

Ответ. Python вызывает `object.__str__()` и печатает строку. Это полиморфизм: у `User`, `Role` и `Group` разные тексты.

13. Что такое конструктор объекта?

Ответ. Метод `__init__`, вызывается при `User(...)`.

14. Можно ли создать несколько объектов одного класса?

Ответ. Да. В `data/users.json` шесть пользователей — шесть объектов `User`.

15. Могут ли объекты одного класса иметь разные значения атрибутов?

Ответ. Да. Иванова — менеджер, Думлер — администратор, хотя оба объекта `User`.

По принципам ООП

16. Что такое инкапсуляция?

Ответ. Данные и операции над ними собраны в объекте. Список прав роли спрятан в `Role._permissions`, снаружи — свойство `permissions` и метод `has_permission`.

17. Как инкапсуляция проявляется в проекте?

Ответ. Нельзя «просто поменять словарь». Чтобы сменить роль, вызывается `user.assign_role(role)`. Проверка права — метод роли.

18. Что такое наследование?

Ответ. Класс-потомок получает атрибуты и методы родителя. `User`, `Role`, `Group`, `Permission` наследуют `Entity` (`id`, `name`, `__str__`).

19. Что означает отношение «является» при наследовании?

Ответ. `User` является `Entity`: у него есть `id` и имя, плюс свои поля. Наследование без такого смысла методичка не рекомендует.

20. Почему не следует использовать наследование без предметного основания?

Ответ. Иначе классы склеиваются искусственно. Я не делала `AdminRole` ради галочки: роли отличаются данными, не типом.

21. Что такое полиморфизм?

Ответ. Одинаковый вызов ведёт себя по-разному у разных классов. `print(obj)` для `User`, `Role` и `Permission` даёт разный текст.

22. Как `__str__()` связан с полиморфизмом?

Ответ. Все сущности — `Entity`, но каждый класс задаёт своё `__str__`. Интерфейс один, представление разное.

23. Что такое композиция?

Ответ. Объект содержит другие объекты. `User` содержит `Role` и `Group`, `Role` содержит список `Permission`.

24. Чем композиция отличается от наследования?

Ответ. Наследование — «является». Композиция — «имеет». Пользователь не является ролью, он имеет роль.

25. Как композиция используется в классе `Booking` / в моём проекте?

Ответ. В примере методички `Booking` хранит `Room` и `User`. У меня `User` хранит объекты `Role` и `Group`, `Role` — объекты `Permission`. В JSON остаются только `id`.

По методам и декораторам

26. Что такое декоратор?

Ответ. Функция, которая принимает другую функцию и возвращает обёртку. В проекте: `@staticmethod`, `@classmethod`, `@property`, `@log_action`.

27. Для чего используется декоратор?

Ответ. Чтобы добавить поведение, не меняя тело функции: превратить метод в статический, создать объект из JSON, сохранить метаданные через `wraps`.

28. Что такое функция-обертка?

Ответ. Внутренняя функция `wrapper` в декораторе. Она вызывает исходную функцию. Так устроен `log_action` в `utils.py`.

29. Для чего используются `*args` и `**kwargs` в декораторах?

Ответ. Чтобы обёртка принимала любые позиционные и именованные аргументы исходной функции.

30. Для чего используется `functools.wraps`?

Ответ. Копирует имя и `docstring` исходной функции на обёртку. Иначе интроспекция видела бы `wrapper`, а не `persist`.

31. Для чего применяется `@staticmethod`?

Ответ. Метод логически относится к классу, но не использует `self`. `Role.check_permission(role, permission)` — правило ПР1. `User.validate_age(age)` — проверка возраста.

32. Чем `staticmethod` отличается от обычного метода?

Ответ. Нет `self`. Вызывается через класс: `Role.check_permission`. Обычный метод нужен, когда важны атрибуты объекта.

33. Для чего используется `@classmethod`?

Ответ. Первый параметр — `cls`, класс. `from_data` создаёт объект из словаря JSON: `User.from_data(data, roles, groups)`.

34. Чем `classmethod` отличается от метода объекта?

Ответ. Работает с классом, а не с уже созданным экземпляром. Удобно как альтернативный конструктор.

35. Для чего используется `@property`?

Ответ. Чтобы обращаться к вычисляемому значению как к полю. `role.access_description` без скобок возвращает текст доступа.

36. Почему `booking.status` может использоваться без круглых скобок?

Ответ. Если это `@property`, вызов выглядит как атрибут. У меня так сделано `access_description` и `permissions`.

По индивидуальному проекту

37. Какие сущности выделены в индивидуальном проекте?

Ответ. Пользователь, роль, разрешение, группа. Плюс базовый Entity для наследования.

38. Почему Room / User представлен классом?

Ответ. У пользователя свои данные, состояние и операции: назначить роль, проверить право, показать карточку. Это объект, а не набор разрозненных функций.

39. Какие атрибуты имеет User?

Ответ. `id`, `name`, `age`, `role` (объект Role), `group` (объект Group или None), `registration_date`.

40. Какие операции относятся непосредственно к User / Role?

Ответ. К User: `assign_role`, `set_group`, `has_permission`. К Role: `has_permission`, `permission_codes`, описание доступа.

41. Почему User хранит объект Role, а не только role_id?

Ответ. Чтобы вызывать `user.role.has_permission` и `user.role.name` без поиска по словарю. `id` остаются только в JSON.

42. Почему User связан с объектом Group?

Ответ. Группа — отдельная сущность. Связь «пользователь состоит в отделе» — композиция, как Booking связан с User в примере.

43. Какие функции ПР2 были преобразованы в методы?

Ответ. `check_permission` → `Role.check_permission`; `describe_role` → `access_description`; назначение роли и группы; проверка права пользователя.

44. Какие функции ПР2 остались обычными функциями и почему?

Ответ. `find_users`, `sort_users`, `load/save JSON`, `input_int` — они работают с коллекцией или файлом, а не с одним объектом.

45. Как выполняется преобразование JSON в объекты?

Ответ. `storage.load_permissions/roles/groups/users` читает JSON и вызывает `from_data`. Для User сначала находятся Role и Group.

46. Как выполняется сохранение объектов в JSON?

Ответ. `save_users` пишет `role_id` и `group_id`, а не сами объекты. Файлы остаются списками словарей.

47. Как изменились коллекции данных по сравнению с ПР2?

Ответ. Было `dict[int, dict]`. Стало `list[User]`, `list[Role]`, `list[Group]`, `list[Permission]`.

48. Какие тесты были адаптированы после перехода к ООП?

Ответ. Все тесты ПР2 переписаны на объекты: создание User/Role, `__str__`, `from_data`, связи, JSON с `role_id`. Сейчас 22 теста.

49. Как проверить, что старая функциональность не потеряна?

Ответ. `pytest -v` — 22 passed. Меню те же пункты. Иванова-менеджер по-прежнему не может удалять пользователей.

50. Какова роль `main.py` после перехода к объектной модели?

Ответ. Только меню и сценарии. Данные — коллекции объектов, создание и поиск — функции `models`, JSON — `storage.py`.

Репозиторий: <https://github.com/VikaD2345/-User-role-management-system>